

Projet Systèmes

Auteurs : LIMOUSY Lilian, AMARA Kousay

UF Informatique, Université de Bordeaux

8 Décembre 2025

Table des matières

1	Introduction	2
2	Partie I. Adressage virtuel par une table de pages	2
2.1	Chargement via <code>ReadAtVirtual</code> (Actions I.2 - I.3)	2
2.2	Gestion de la mémoire physique : <code>PageProvider</code> (Action I.5)	2
3	Partie II. Exécuter plusieurs programmes en même temps	2
3.1	L'appel système <code>ForkExec</code> (II.1)	2
3.2	Raffinage (II.2)	3
3.3	Processus et Threads multiples (II.3)	3
3.4	Tests (II.4)	3
3.5	Mélange des threads et <code>Exit</code> (II.5 & II.6)	3
3.6	Analyse des fuites mémoire (Bonus Action II.7)	3
4	Conclusion	3

1 Introduction

L'objectif de ce devoir était de permettre à Nachos d'exécuter plusieurs programmes simultanément (multiprogrammation) et de gérer la mémoire via une table des pages (pagination). Nous avons également sécurisé les accès concurrents et implémenté une gestion robuste de la terminaison des processus.

2 Partie I. Adressage virtuel par une table de pages

Cette première partie visait à séparer l'adressage virtuel de l'adressage physique pour permettre un chargement flexible des programmes en mémoire.

2.1 Chargement via `ReadAtVirtual` (Actions I.2 - I.3)

Le chargement initial des exécutables NOFF écrivait directement en mémoire physique. Pour supporter la pagination, nous avons implémenté la fonction `ReadAtVirtual`.

- Cette fonction lit les données du fichier dans un tampon temporaire.
- Elle écrit ensuite octet par octet en mémoire via `machine->WriteMem`, ce qui force l'utilisation de la traduction d'adresse (TLB/PageTable).
- **Point délicat** : Il a fallu sauvegarder et restaurer `machine->currentPageTable` pour s'assurer que l'écriture se fasse bien dans l'espace d'adressage en cours de construction.

2.2 Gestion de la mémoire physique : `PageProvider` (Action I.5)

Pour gérer les pages physiques disponibles, nous avons créé la classe singleton `PageProvider`.

- Elle utilise un `BitMap` pour suivre l'état des pages (libres/occupées).
- La méthode `GetEmptyPage` retourne une page libre et l'initialise à zéro (`memset`) pour éviter les fuites de données entre processus.
- La méthode `ReleasePage` libère la page lors de la destruction d'un `AddrSpace`.

3 Partie II. Exécuter plusieurs programmes en même temps

L'objectif principal était de conserver dans la mémoire plusieurs programmes en même temps.

3.1 L'appel système `ForkExec` (II.1)

Nous avons implémenté l'appel système `ForkExec` qui permet à un programme de lancer un nouvel exécutable dans un espace d'adressage séparé.

- Le noyau ouvre l'exécutable et crée un nouvel `AddrSpace`.
- Un nouveau thread noyau est créé et initialisé avec cet espace.
- La fonction `StartUserProg` initialise les registres et lance la machine pour le nouveau processus.

Il ne faut pas oublier d'incrémenter le compteur de processus avant de lancer le thread pour éviter qu'il se termine avant que le thread parent n'ait enregistré sa naissance.

3.2 Raffinage (II.2)

Nous avons modifié `SC_Exit`. Au lieu d'arrêter la machine systématiquement, `SC_Exit` décrémente un compteur global `processCount`.

- Si le compteur est > 0 , seul le thread courant se termine (`Finish`).
- Si le compteur atteint 0, c'est le dernier processus : on arrête tout comme avant.

3.3 Processus et Threads multiples (II.3)

Nous avons adapté `AddrSpace` pour supporter plusieurs threads par processus.

- L'allocation des piles est gérée par un `BitMap` dans `AddrSpace`, permettant d'attribuer des slots de pile uniques à chaque thread.
- Nous avons augmenté `UserStacksAreaSize` à 4096 octets et `NumPhysPages` à 512 pour supporter la charge mémoire.

3.4 Tests (II.4)

Nous avons réalisé un test de stress (`dozen_threads_launcher`) lançant **12 processus**, chacun créant **12 threads** (Total : 144 threads).

- Le système reste stable.
- Les verrous dans `ConsoleDriver` fonctionnent donc il n'y a pas eu d'anomalie.

3.5 Mélange des threads et Exit (II.5 & II.6)

Liste des threads (II.5) : Nous avons ajouté une liste `threadList` dans `AddrSpace`. Lorsqu'un thread appelle `Exit`, il détruit l'espace d'adressage (`delete space`), ce qui invalide la mémoire pour tous les autres threads du processus, les forçant à s'arrêter.

Le verrou dans la Console (II.6) : Si un thread est tué alors qu'il détient le verrou de la console, tout le système peut bloquer. Nous avons implémenté une méthode `ClearConsoleLocks` qui force la libération du verrou si son propriétaire est détruit.

3.6 Analyse des fuites mémoire (Bonus Action II.7)

L'outil `LeakSanitizer` rapporte "1344136 byte(s) leaked in 117 allocation(s)" lors du test. Il râle pour la pile du dernier thread noyau en cours d'exécution car il appelle `interrupt->Powerdown` avant de se free.

4 Conclusion

Nachos accepte maintenant plusieurs processus en même temps, chacun possédant un ou plusieurs thread, le tout avec une bonne gestion mémoire (pagination, synchronisation) même en en ayant beaucoup !